



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

ÁREA MECÁNICA ELÉCTRICA



PROYECTO PARCIAL 3

TRATAMIENTO DE IMÁGENES

ALUMNO: RAMÍREZ GALVÁN MANUEL

CLAVE: 288093

PROFESOR: JULLY KADO MERCADO ELIAS

FECHA: 02/ABRIL/2025

Contenido

Introducción.....	3
Desarrollo.....	3
Explicación del Código para recortar Piezas.....	3
Explicación del Código para crear Tablero.....	5
Explicación del Código para colocar Piezas en el Tablero.....	5
Código para Generar Piezas.....	8
Código para Generar Tablero.....	8
Código para hacer Rompecabezas.....	9
Imágenes Utilizadas.....	10
Resultados.....	11
Conclusiones.....	12
Referencias.....	12

Introducción

El procesamiento de imágenes y la visión por computadora permiten hacer la reconstrucción de imágenes a partir de fragmentos, como ocurre en el armado de un rompecabezas.

El problema es reconstruir una imagen original a partir de piezas previamente recortadas. A diferencia de enfoques deterministas, donde se conoce de antemano la posición exacta de cada pieza, aquí se busca una solución completamente automática basada en el análisis de características visuales.

Para lograrlo, se emplea el algoritmo ORB (Oriented FAST and Rotated BRIEF), que permite detectar puntos clave y calcular descriptores robustos incluso ante variaciones de escala o rotación. Estos descriptores se comparan entre cada pieza y el tablero incompleto, y mediante el uso de la homografía se calcula la transformación necesaria para colocar cada fragmento en su posición correspondiente.

Este enfoque simula un escenario más realista, donde se desconoce el lugar exacto de las piezas, y se pone a prueba la **capacidad del algoritmo para encontrar coincidencias confiables** basadas únicamente en la información visual.

Desarrollo

Explicación del Código para recortar Piezas

Se coloca la librería de OpenCV para el procesamiento de imágenes.

```
1 import cv2
```

Ilustración 1

Se carga la imagen original donde se van a extraer las piezas.

```
3 # Cargar la imagen base
4 imagen = cv2.imread("Foto.jpg")
```

Ilustración 2

Se redimensiona la imagen a un tamaño fijo para asegurar que las coordenadas que se usarán para recortar las piezas correspondan con el tablero el cual tiene que tener el mismo redimensionamiento.

```
6 # Redimensionar
7 imagen = cv2.resize(imagen, (600, 800))
```

Ilustración 3

Coordenadas en X y en Y para cada una de las imágenes, además del ancho y alto de cada una de ellas.

```
9 # Coordenadas (x, y, ancho, alto)
10 coordenadas_piezas = [
11     (150, 250, 150, 150),
12     (300, 50, 150, 150),
13     (50, 550, 150, 150),
14     (450, 300, 150, 150),
15 ]
```

Ilustración 4

Se extrae una imagen de la original con la coordenada y tamaño en cada una de ellas.

```
17 for i, (x, y, w, h) in enumerate(coordenadas_piezas):
18     # Recortar la pieza
19     pieza = imagen[y:y+h, x:x+w]
```

Ilustración 5

Se rota la primera imagen 90° a la derecha, a la segunda se le coloca un filtro Gaussiano, a la tercera se le reduce su tamaño en un 80% y la cuarta tiene una rotación de 45°

```
21 # Aplicar una transformación diferente a cada pieza
22 if i == 0:
23     pieza = cv2.rotate(pieza, cv2.ROTATE_90_CLOCKWISE)
24 elif i == 1:
25     pieza = cv2.GaussianBlur(pieza, (5, 5), 0)
26 elif i == 2:
27     pieza = cv2.resize(pieza, (int(w*0.8), int(h*0.8)))
28 elif i == 3:
29     matriz_rot = cv2.getRotationMatrix2D((w//2, h//2), 45, 1)
30     pieza = cv2.warpAffine(pieza, matriz_rot, (w, h))
```

Ilustración 6

Se guarda cada una de las piezas.

```
32 # Guardar la pieza
33 cv2.imwrite(f"pieza_{i+1}.png", pieza)
```

Ilustración 7

Explicación del Código para crear Tablero

Esta parte es el mismo código que para recortar las piezas.

```
1  import cv2
2
3  # Cargar la imagen original
4  imagen = cv2.imread("Foto.jpg")
5
6  # Redimensionar
7  imagen = cv2.resize(imagen, (600, 800))
8
9  # Coordenadas
10 coordenadas_piezas = [
11     (150, 250, 150, 150),
12     (300, 50, 150, 150),
13     (50, 550, 150, 150),
14     (450, 300, 150, 150),
15 ]
```

Ilustración 8

Crear una copia de la imagen original para no modificar la original.

```
17 # Crear una copia para no modificar la original
18 tablero = imagen.copy()
```

Ilustración 9

Se dibuja un cuadro blanco relleno para simular un hueco en el tablero con las mismas coordenadas y tamaños que el recorte de piezas.

```
20 # Dibujar los huecos en blanco
21 for (x, y, w, h) in coordenadas_piezas:
22     cv2.rectangle(tablero, (x, y), (x+w, y+h), (255, 255, 255), -1)
```

Ilustración 10

Explicación del Código para colocar Piezas en el Tablero

Se coloca la librería de OpenCV, NumPy y Matplotlib para trabajar con ellas en manipulación y procesamiento de las imágenes, operaciones con números y matrices de imágenes e impresión adecuada de las imágenes en ventanas.

```
1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
```

Ilustración 11

Se carga el tablero con huecos y se redimensiona, además de convertirla en escala de grises para facilitar la detección de puntos clave.

```
# Cargar el tablero con huecos
tablero = cv2.imread("tablero_con_huecos.png")
tablero = cv2.resize(tablero, (600, 800))
tablero_gray = cv2.cvtColor(tablero, cv2.COLOR_BGR2GRAY)
```

Ilustración 12

Se utiliza ORB para detectar puntos clave y calcular descriptores; BFMatcher se utiliza para emparejar los descriptores entre la pieza y el tablero.

```
10 # Inicializar ORB y matcher
11 orb = cv2.ORB_create(nfeatures=500)
12 bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
```

Ilustración 13

Se utiliza un ciclo para repetirlo en las 4 piezas

```
14 n_piezas = 4
15
16 for i in range(1, n_piezas + 1):
```

Ilustración 14

Se carga cada una de las piezas, se ajusta su tamaño y convertirlas en escala de grises.

```
17 pieza = cv2.imread(f"pieza_{i}.png")
18 pieza = cv2.resize(pieza, (150, 150))
19 pieza_gray = cv2.cvtColor(pieza, cv2.COLOR_BGR2GRAY)
```

Ilustración 15

Se detectan los puntos clave y los descriptores, ya que ORB detecta estos puntos en las imágenes y extrae los descriptores para cada uno.

```
21 # Detectar keypoints y descriptores
22 kp1, des1 = orb.detectAndCompute(pieza_gray, None)
23 kp2, des2 = orb.detectAndCompute(tablero_gray, None)
```

Ilustración 16

Se buscan las mejores coincidencias entre los puntos clave de la pieza y el tablero y se ordenan los matches por similitud.

```
29 matches = bf.match(des1, des2)
30 matches = sorted(matches, key=lambda x: x.distance)
```

Ilustración 17

Creación, visualización y guardado de las imágenes con los matches.

```
36 # Visualizar los matches
37 img_matches = cv2.drawMatches(pieza, kp1, tablero, kp2, matches[:30], None, flags=2)
38 plt.figure(figsize=(25, 10))
39 plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
40 plt.title(f"Matches de la Pieza {i}")
41 plt.axis("off")
42 plt.savefig(f"matches_pieza_{i}.png")
43 plt.close()
```

Ilustración 18

Se utilizan los 30 mejores matches para calcular la matriz de homografía.

```
45 # Homografía con los mejores matches
46 src_pts = np.float32([kp1[m.queryIdx].pt for m in matches[:30]]).reshape(-1, 1, 2)
47 dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches[:30]]).reshape(-1, 1, 2)
48
49 H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

Ilustración 19

Se transforma la pieza según el cálculo de la homografía.

```
56 # Transformar la pieza
57 pieza_warp = cv2.warpPerspective(pieza, H, (tablero.shape[1], tablero.shape[0]))
```

Ilustración 20

Se crea una máscara para colocar la parte transformada de la pieza y colocarla en el tablero.

```
58 mascara = cv2.warpPerspective(np.ones((h_p, w_p), dtype=np.uint8) * 255, H, (tablero.shape[1], tablero.shape[0]))
59
60 # Colocar la pieza en el tablero
61 tablero[mascara > 0] = pieza_warp[mascara > 0]
```

Ilustración 21

Dibujo del contorno alrededor de la pieza colocada y la colocación de su etiqueta.

```
63 # Dibujar el contorno de la pieza colocada
64 esquinas = np.float32([[0, 0], [w_p, 0], [w_p, h_p], [0, h_p]]).reshape(-1, 1, 2)
65 esquinas_transformadas = cv2.perspectiveTransform(esquinas, H).astype(int)
66 color = tuple(int(c) for c in np.random.randint(0, 255, size=3))
67 cv2.polylines(tablero, [esquinas_transformadas], isClosed=True, color=color, thickness=3)
68 cv2.putText(tablero, f"Pieza {i}",
69             tuple(esquinas_transformadas[0][0]),
70             cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
```

Ilustración 22

Visualización y guardado de cada paso al colocar las piezas.

```
72 # Mostrar y guardar el paso
73 cv2.imshow(f"Paso {i}", tablero)
74 cv2.imwrite(f"paso_{i}_orb.png", tablero)
75 print(f"Pieza {i} colocada. Guardada como paso_{i}_orb.png")
76 cv2.waitKey(0)
77 cv2.destroyAllWindows(f"Paso {i}")
```

Ilustración 23

Visualización y guardado del resultado final.

```
79 # Guardar resultado final
80 cv2.imwrite("rompecabezas_final_orb.png", tablero)
81 cv2.imshow("Rompecabezas Final (ORB)", tablero)
82 cv2.waitKey(0)
83 cv2.destroyAllWindows()
```

Ilustración 24

Código para Generar Piezas

```
import cv2

# Cargar la imagen base
imagen = cv2.imread("Foto.jpg")

# Redimensionar por si está muy grande
imagen = cv2.resize(imagen, (600, 800))

# Crear lista de coordenadas para 4 piezas (x, y, ancho, alto)
coordenadas_piezas = [
    (150, 250, 150, 150),
    (300, 50, 150, 150),
    (50, 550, 150, 150),
    (450, 300, 150, 150),
]

for i, (x, y, w, h) in enumerate(coordenadas_piezas):
    # Recortar la pieza
    pieza = imagen[y:y+h, x:x+w]

    # Aplicar una transformación diferente a cada pieza
    if i == 0:
        pieza = cv2.rotate(pieza, cv2.ROTATE_90_CLOCKWISE)
    elif i == 1:
        pieza = cv2.GaussianBlur(pieza, (5, 5), 0)
    elif i == 2:
        pieza = cv2.resize(pieza, (int(w*0.8), int(h*0.8)))
    elif i == 3:
        matriz_rot = cv2.getRotationMatrix2D((w//2, h//2), 45, 1)
        pieza = cv2.warpAffine(pieza, matriz_rot, (w, h))

    # Guardar la pieza
    cv2.imwrite(f"pieza_{i+1}.png", pieza)
```

Ilustración 25 Código Piezas

Código para Generar Tablero

```
import cv2

# Cargar la imagen original
imagen = cv2.imread("Foto.jpg")

# Redimensionar
imagen = cv2.resize(imagen, (600, 800))

# Coordenadas
coordenadas_piezas = [
    (150, 250, 150, 150),
    (300, 50, 150, 150),
    (50, 550, 150, 150),
    (450, 300, 150, 150),
]

# Crear una copia para no modificar la original
tablero = imagen.copy()

# Dibujar los huecos en blanco
for (x, y, w, h) in coordenadas_piezas:
    cv2.rectangle(tablero, (x, y), (x+w, y+h), (255, 255, 255), -1)

# Guardar la imagen del tablero
cv2.imwrite("tablero_con_huecos.png", tablero)
```

Ilustración 26 Código Tablero

Código para hacer Rompecabezas

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar el tablero con huecos
tablero = cv2.imread("tablero_con_huecos.png")
tablero = cv2.resize(tablero, (600, 800))
tablero_gray = cv2.cvtColor(tablero, cv2.COLOR_BGR2GRAY)

# Inicializar ORB y matcher
orb = cv2.ORB_create(nfeatures=500)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

n_piezas = 4

for i in range(1, n_piezas + 1):
    pieza = cv2.imread(f"pieza_{i}.png")
    pieza = cv2.resize(pieza, (150, 150))
    pieza_gray = cv2.cvtColor(pieza, cv2.COLOR_BGR2GRAY)

    # Detectar keypoints y descriptores
    kp1, des1 = orb.detectAndCompute(pieza_gray, None)
    kp2, des2 = orb.detectAndCompute(tablero_gray, None)

    if des1 is None or des2 is None or len(kp1) < 4 or len(kp2) < 4:
        print(f"Pieza {i} no tiene suficientes puntos clave.")
        continue

    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda x: x.distance)

    if len(matches) < 10:
        print(f"No hay suficientes matches para la pieza {i}.")
        continue

    # Visualizar los matches
    img_matches = cv2.drawMatches(pieza, kp1, tablero, kp2, matches[:30], None, flags=2)
    plt.figure(figsize=(25, 10))
    plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
    plt.title(f"Matches de la Pieza {i}")
    plt.axis("off")
    plt.savefig(f"matches_pieza_{i}.png")
    plt.close()

    # Homografía con los mejores matches
    src_pts = np.float32([kp1[m.queryIdx].pt for m in matches[:30]]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches[:30]]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    if H is None:
        print(f"No se pudo calcular homografía para pieza {i}.")
        continue

    h_p, w_p = pieza.shape[:2]

    # Transformar la pieza
    pieza_warp = cv2.warpPerspective(pieza, H, (tablero.shape[1], tablero.shape[0]))
    mascara = cv2.warpPerspective(np.ones((h_p, w_p), dtype=np.uint8) * 255, H, (tablero.shape[1],
    tablero.shape[0]))

    # Colocar la pieza en el tablero
    tablero[mascara > 0] = pieza_warp[mascara > 0]

    # Dibujar el contorno de la pieza colocada
    esquinas = np.float32([[0, 0], [w_p, 0], [w_p, h_p], [0, h_p]]).reshape(-1, 1, 2)
    esquinas_transformadas = cv2.perspectiveTransform(esquinas, H).astype(int)
    color = tuple(int(c) for c in np.random.randint(0, 255, size=3))
    cv2.polylines(tablero, [esquinas_transformadas], isClosed=True, color=color, thickness=3)
    cv2.putText(tablero, f"Pieza {i}",
                tuple(esquinas_transformadas[0][0]),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

    # Mostrar y guardar el paso
    cv2.imshow(f"Paso {i}", tablero)
    cv2.imwrite(f"paso_{i}_orb.png", tablero)
    print(f"Pieza {i} colocada. Guardada como paso_{i}_orb.png")
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# Guardar resultado final
cv2.imwrite("rompecabezas_final_orb.png", tablero)
cv2.imshow("Rompecabezas Final (ORB)", tablero)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Ilustración 27 Código Tablero

Imágenes Utilizadas

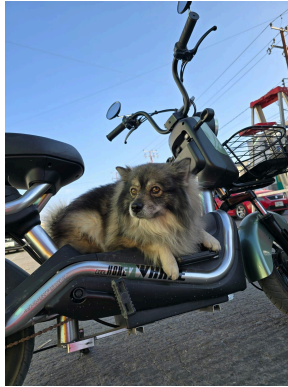


Ilustración 28 Imagen Utilizada

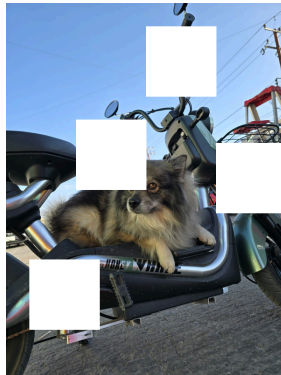


Ilustración 29 Tablero Generado



Ilustración 29 Piezas Generadas

Resultados

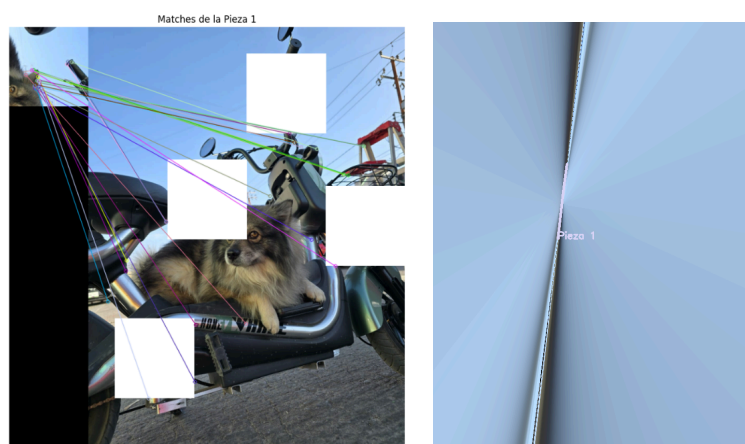


Ilustración 30 Paso 1

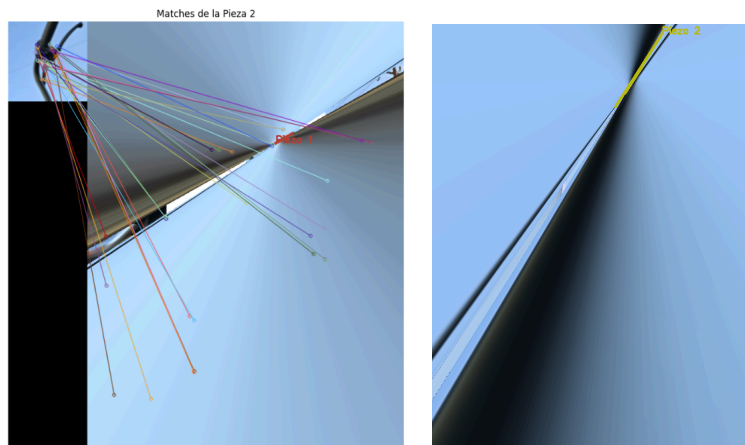


Ilustración 31 Paso 2

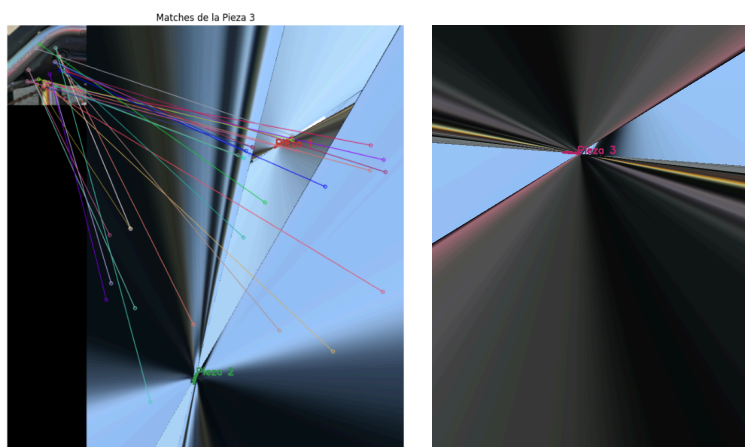


Ilustración 32 Paso 3

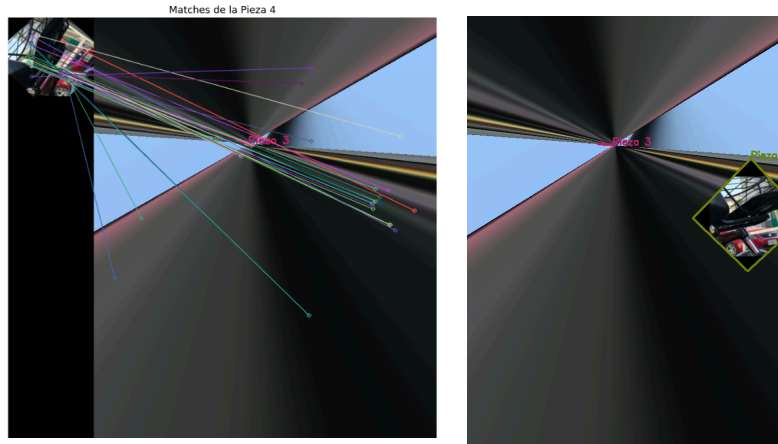


Ilustración 33 Paso 4

Conclusiones

Se implementó un sistema de reconstrucción de imágenes tipo rompecabezas utilizando, específicamente con el detector de puntos clave ORB y el cálculo de homografía. La intención fue evaluar la capacidad del algoritmo para colocar automáticamente piezas de una imagen en su lugar original, sin conocimiento previo de sus posiciones.

Sin embargo, los resultados obtenidos no fueron los esperados. El análisis visual mostró una gran dispersión en los puntos coincidentes entre cada pieza y el tablero. Esta dispersión afectó directamente el cálculo de la matriz de homografía, lo cual provocó transformaciones incorrectas al aplicar `warpPerspective`.

El fallo se hizo evidente mediante el efecto característico de homografía mal aplicada, conocido como efecto túnel o warp colapsado, donde la pieza se deforma de forma extrema hacia un punto, provocando un resultado visual irreconocible. En las pruebas realizadas, solo una de las cuatro piezas fue colocada correctamente, mientras que el resto generó transformaciones erróneas debido a la inadecuada distribución de los matches.

Estos resultados reflejan que, aunque el método automático con ORB y homografía tiene un gran potencial, su funcionamiento correcto depende de varios factores críticos:

Se concluye que, para obtener buenos resultados con este método, es necesario asegurar condiciones visuales óptimas o bien implementar filtros adicionales que permitan descartar emparejamientos erróneos antes de aplicar la homografía.

Referencias

Ramírez Galván, M. (2025). Actividades de procesamiento de imágenes mediante técnicas de filtrado y segmentación (P3_A1, P3_A3, P3_A3, P3_A4). Universidad Autónoma de San Luis Potosí.

ChatGPT